

Smart Ride

Developer Documentation

Multi-Service Mobility Platform

Onboarding Guide for New Developers

Table of Contents

1. What is Smart Ride?
2. Project Structure
3. Tech Stack
4. Database Models
5. API Endpoints
6. Approval Workflows
7. Authentication Flow
8. Real-time Features
9. Mobile App Architecture
10. External Integrations
11. Quick Start Commands
12. Important Notes

1. What is Smart Ride?

Smart Ride is a multi-service mobility platform in Uganda offering:

- Smart Boda - Motorcycle taxi rides
- Smart Car - Car rides
- Food Delivery - Restaurant ordering
- Shopping Delivery - Supermarket/retail
- Item Delivery - Parcel/package delivery
- Smart Health - Pharmacy delivery with prescription verification

2. Project Structure

```
/home/z/my-project/  
src/ # Main Next.js application  
app/ # Pages and API routes (App Router)  
api/ # REST API endpoints  
admin/ # Admin dashboard pages  
page.tsx # Home page  
components/  
ui/ # 50+ shadcn/ui components  
dashboard/ # Admin dashboard components  
smart-ride/ # Main app components  
mobile/ # Mobile-specific screens  
lib/ # Core services and utilities  
prisma/ # Database schema  
smart-ride-mobile/ # React Native/Expo mobile app  
mini-services/ # Microservices (Socket.io, etc.)  
db/ # SQLite database
```

3. Tech Stack

Layer	Technology
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS 4
UI Library	shadcn/ui (50+ components)
State	Zustand (client), TanStack Query (server)

Database	Prisma ORM + SQLite
Auth	JWT + Firebase (Google/Phone)
Real-time	Socket.io
Maps	Mapbox
Mobile	Expo (React Native) + NativeWind

4. Database Models (Key Entities)

User - Base user model for all roles

Rider - Service providers (drivers)

Vehicle - Rider vehicle information

Merchant - Restaurants, shops, pharmacies

MenuItem - Merchant product catalog

Order - Food/shopping orders

Task - Unified model for all services

Payment - Payment records

HealthProvider - Pharmacies, clinics, doctors

HealthOrder - Smart Health pharmacy orders

Prescription - Medical prescriptions

SOSAlert - Emergency alerts

Task Lifecycle (State Machine)

CREATED - MATCHING - ASSIGNED - ACCEPTED - ARRIVED - PICKED_UP - IN_TRANSIT - DELIVERED - COMPLETED

(or CANCELLED at any point)

5. API Endpoints Overview

Route	Purpose
/api/auth/*	Login, register, OAuth, token refresh
/api/tasks/*	Create rides/deliveries, track, update status
/api/orders/*	Food/shopping orders
/api/riders/*	Rider management, approval
/api/merchants/*	Restaurant/shop management

/api/health-provider/*	Pharmacy registration, verification
/api/health-orders/*	Medicine delivery orders
/api/dispatch/*	Task matching and assignment
/api/sos/*	Emergency alerts
/api/payments/*	Payment processing
/api/admin/*	Admin operations

6. Approval Workflows (CRITICAL)

All service providers require admin verification:

Riders

Endpoint: POST /api/riders/approve

Status: PENDING_APPROVAL - APPROVED/REJECTED/SUSPENDED

Equipment tracked: hasReflectorVest, hasHelmet, hasInsulatedBox

Merchants

Endpoint: POST /api/admin/merchants/verify

Status: PENDING_APPROVAL - APPROVED/REJECTED/SUSPENDED

Actions: approve, reject, suspend, activate

Health Providers

Endpoint: POST /api/admin/health-providers/verify

Status: PENDING - APPROVED/REJECTED/SUSPENDED/DOCUMENTS_REQUESTED

Actions: approve, reject, suspend, activate, request_documents

Entity	Initial Status	Approval Endpoint	Required Role
Rider	PENDING_APPROVAL	/api/riders/approve	ADMIN
Merchant	PENDING_APPROVAL	/api/admin/merchants/verify	ADMIN
Health Provider	PENDING	/api/admin/health-providers/verify	ADMIN

7. Authentication Flow

Email/Password Login - JWT Token

Google OAuth - Firebase - JWT Token

Phone OTP - Firebase - JWT Token

User Roles

CLIENT - Regular users

RIDER - Drivers (boda/car/delivery)

MERCHANT - Restaurant/shop owners

PHARMACIST / HEALTH_PROVIDER - Pharmacy staff

ADMIN, SUPER_ADMIN, etc. - Platform management

8. Real-time Features (Socket.io)

Port 3001 - WebSocket server

Key Events:

task:status - Live task updates

rider:location - GPS tracking

dispatch:request - New task notification to driver

order:status - Order updates

chat:message - In-app messaging

sos:alert - Emergency broadcast

heartbeat - Connection health

9. Mobile App Architecture

smart-ride-mobile/

app/ # Expo Router screens
(tabs)/ # Tab navigation
auth/ # Login, Register
rider/ # Ride booking flow
driver/ # Driver dashboard
orders/ # Food/order screens
src/
store/ # Zustand stores
services/ # API client, Socket.io
config/ # Environment config

10. External Integrations

Firestore

Google Sign-In, Phone Authentication (SMS OTP), FCM Push Notifications

Mapbox

Geocoding (Uganda-focused), Reverse geocoding, Directions API, Static map generation

Environment Variables Needed

```
DATABASE_URL - SQLite database path
JWT_SECRET - Authentication secret
NEXT_PUBLIC_MAPBOX_ACCESS_TOKEN - Mapbox token
NEXT_PUBLIC_FIREBASE_* - Firebase config keys
```

11. Quick Start Commands

```
# Install dependencies
bun install
```

```
# Setup database
bun run db:push
bun run db:seed
```

```
# Run web app (port 3000)
bun run dev
```

```
# Run Socket.io service (port 3001)
cd mini-services/realtime-service && bun run index.ts
```

```
# Run mobile app (local development)
cd smart-ride-mobile && npx expo start
```

12. Important Notes

1. Riders need admin approval before accessing driver features
2. Merchants need admin approval before receiving orders
3. Health Providers need admin approval before processing prescriptions
4. Admin dashboard NOT available in mobile app - Role-based restriction

5. All prices in UGX (Ugandan Shilling)
6. Mapbox configured for Uganda - Kampala default coordinates
7. Real-time features require Socket.io service running

Generated by Smart Ride Development Team